From Matchings to Max-Flow Min-Cut

Network Flow Theory and Applications

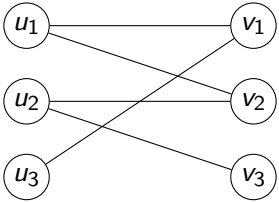
Daniel Mortensen

April 8, 2026

Outline

Bipartite Graphs

A **bipartite graph** $G = (U \cup V, E)$ has two disjoint vertex sets with edges only between them.



Question: How many independent pairs can we match?

Matchings

A **matching** $M \subseteq E$ is a set of edges with no shared endpoints.

- A vertex is **matched** if it is an endpoint of some edge in M; otherwise **free**.
- A **maximum matching** has the largest possible $|M|$.
- A **perfect matching** matches every vertex ($|M| = |U| = |V|$).

Goal: Find a maximum matching efficiently.

Alternating and Augmenting Paths

Given a matching M:

- An **alternating path** alternates between edges in M and edges not in M.
- An **augmenting path** is an alternating path that starts and ends at free (unmatched) vertices.

Key insight: If an augmenting path exists, we can *flip* all edges along it — matched $\leftrightarrow$ unmatched — to get a matching of size $|M| + 1$.

Theorem (Berge, 1957)

A matching M is maximum if and only if there is no augmenting path with respect to M.

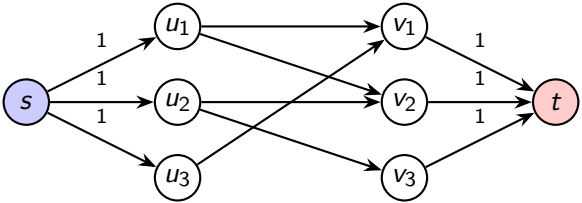
Finding Maximum Matchings via Augmenting Paths

```
Start with any matching M (e.g. M = ∅)
repeat
    Search for an augmenting path P (BFS/DFS from free vertices)
    if P found then
        Augment: M ← M ⊕ P (symmetric difference)
    end if
until no augmenting path exists
return M (maximum matching)
```

Each augmentation increases $|M|$ by 1, so at most $|V|/2$ iterations.

Adding a Source and Sink

A bipartite matching problem is a **network flow** problem in disguise.



- Add source s with edges to all of U (capacity 1).
- Add sink t with edges from all of V (capacity 1).
- Original edges get capacity 1.
- Max flow = max matching.

Applications

Network flow models appear everywhere:

- **Bipartite matching**: job assignments, course scheduling
- **Transportation**: shipping goods through a network
- **Telecommunications**: routing data through links with bandwidth limits
- **Image segmentation**: min-cut separates foreground from background
- **Baseball elimination**: can a team still win the league?
- **Our problem**: assigning components to lines

Flow Networks

Definition (Network)

A **network** is a directed graph $G = (V, E)$ with a non-negative capacity function $c : E \to \mathbb{R}_{\geq 0}$, without multiple arcs. WLOG, if $(u, v) \in E$ then $(v, u) \in E$ (add it with $c(v, u) = 0$ if needed).

Definition (Flow network)

A **flow network** (G, c, s, t) distinguishes a **source** s and a **sink** t in G.

Flows

Definition (Flow)

A **flow** is a function $f : E \to \mathbb{R}_{\geq 0}$ satisfying:

1. **Capacity constraint**: $0 \leq f(u,v) \leq c(u,v)$ for all $(u,v) \in E$.
2. **Conservation**: for all $v \in V \setminus \{s,t\}$:

$$\sum_{u:(u,v)\in E} f(u,v) = \sum_{w:(v,w)\in E} f(v,w).$$

The **value** of a flow is $|f| = \sum_{v:(s,v)\in E} f(s,v) - \sum_{u:(u,s)\in E} f(u,s)$.

Cuts

Definition (s-t cut)

An s-t **cut** (S, T) is a partition $V = S \cup T$ with $s \in S$ and $t \in T$.

The **capacity** of the cut:

$$c(S, T) = \sum_{\substack{u \in S,\, v \in T \\ (u,v) \in E}} c(u, v).$$

Intuition: a cut is a "bottleneck" — the maximum flow that can cross from the s-side to the t-side.

General Graphs

Nothing in the flow/cut definitions requires a bipartite structure.

- The graph can have arbitrary topology — cycles, dense subgraphs, etc.
- Capacity constraints and flow conservation apply identically.
- The max-flow min-cut theorem holds for *any* directed graph.

Bipartite matching is a *special case* (unit capacities, layered structure). The general theory is strictly more powerful.

Statement

Theorem (Ford–Fulkerson, 1956)

In any flow network (G, c, s, t)*:*

$$\max_f |f| = \min_{(S,T)} c(S, T).$$

The maximum value of an s-t flow equals the minimum capacity of an s-t cut.

Three equivalent conditions:

1. f is a maximum flow.
2. There is no augmenting path in the residual graph G_f.
3. $|f| = c(S, T)$ for some cut (S, T).

Residual Graph

Given flow f, the **residual graph** $G_f = (V, E_f)$ has:

- **Forward edge** (u, v) with residual capacity $c_f(u, v) = c(u, v) - f(u, v)$ if $c_f(u, v) > 0$.
- **Backward edge** (v, u) with residual capacity $c_f(v, u) = f(u, v)$ if $f(u, v) > 0$.

An **augmenting path** is any $s \to t$ path in G_f.

Key: backward edges allow the algorithm to "undo" previously pushed flow — the same idea as augmenting paths in matching.

Proof Sketch (Weak Duality)

Claim: For any flow f and any cut (S, T): $|f| \leq c(S, T)$.

Proof:

$$
\begin{aligned}
|f| &= \sum_{\substack{u \in S \\ v \in T}} f(u, v) - \sum_{\substack{u \in T \\ v \in S}} f(u, v) \\
&\leq \sum_{\substack{u \in S \\ v \in T}} f(u, v) \leq \sum_{\substack{u \in S \\ v \in T}} c(u, v) = c(S, T).
\end{aligned}
$$

Flow across a cut cannot exceed the cut's capacity.

Proof Sketch (Strong Duality)

Claim: Equality is achieved — there exist f^* and (S^*, T^*) with $|f^*| = c(S^*, T^*)$.

Proof:

1. Run Ford–Fulkerson: repeatedly find augmenting paths in G_f and push flow until no path exists.
2. When no augmenting path exists, define:

$$S^* = \{v \in V : v \text{ reachable from } s \text{ in } G_f\}, \quad T^* = V \setminus S^*.$$

3. Since $t \notin S^*$ (no s-t path in G_f), (S^*, T^*) is a valid cut.
4. For every edge (u, v) with $u \in S^*$, $v \in T^*$: $f(u, v) = c(u, v)$ (otherwise v would be reachable).
5. For every edge (v, u) with $v \in T^*$, $u \in S^*$: $f(v, u) = 0$ (otherwise v would be reachable via backward edge).
6. Therefore $|f^*| = c(S^*, T^*)$. □

Ford–Fulkerson Algorithm

```
Initialise f(u, v) ← 0 for all (u, v) ∈ E
while ∃ augmenting path P in G_f do
    δ ← min_{(u,v)∈P} c_f(u, v)                ▷ bottleneck capacity
    for each edge (u, v) ∈ P do
        if (u, v) is a forward edge then
            f(u, v) ← f(u, v) + δ
        else                                   ▷ backward edge
            f(v, u) ← f(v, u) − δ
        end if
    end for
end while
return f
```

Edmonds–Karp variant: use BFS to find shortest augmenting path $\Rightarrow O(VE^2)$.

Example: Step by Step

(Walk through a small network on the board/slides, showing residual graphs at each iteration.)

1. Start with zero flow.
2. Find augmenting path, push flow.
3. Update residual graph.
4. Repeat until no augmenting path exists.
5. Read off min cut from final residual graph.

Hall's Theorem

Theorem (Hall, 1935)

A bipartite graph $G = (U \cup V, E)$ has a matching that saturates every vertex in U if and only if for every $S \subseteq U$:

$$|N(S)| \geq |S|,$$

where $N(S)$ is the set of neighbours of S in V.

Connection: Hall's condition is equivalent to saying that no s-t cut has capacity less than $|U|$ in the corresponding flow network.

Birkhoff–von Neumann Theorem

Theorem (Birkhoff, 1946; von Neumann, 1953)

Every doubly stochastic matrix is a convex combination of permutation matrices.

- A **doubly stochastic matrix**: non-negative, rows and columns each sum to 1.
- Equivalently: the set of doubly stochastic matrices is the convex hull of permutation matrices.
- **Connection**: fractional matchings (LP relaxation) decompose into integer matchings — the LP relaxation of bipartite matching has no integrality gap.

Kőnig–Egerváry Theorem

Theorem (Kőnig, 1931; Egerváry, 1931)

In a bipartite graph, the size of a maximum matching equals the size of a minimum vertex cover.

- A **vertex cover**: a set of vertices that touches every edge.
- This is the LP-dual relationship: matching $\leq$ vertex cover always, and in bipartite graphs equality holds.
- **Connection**: the min vertex cover corresponds to a min cut in the flow network.

Menger's Theorem

Theorem (Menger, 1927)

The maximum number of vertex-disjoint s-t paths equals the minimum number of vertices (other than s, t) whose removal disconnects s from t.

- Edge-disjoint version: max edge-disjoint paths = min edge cut.
- **Connection**: Menger's theorem is a direct corollary of max-flow min-cut (unit capacities).
- The unifying idea: *the maximum throughput equals the minimum bottleneck*.

Summary

Result	**Says**
Max-flow min-cut	max throughput = min bottleneck
Hall's theorem	perfect matching $\iff$ no small neighbourhood
Birkhoff–von Neumann	fractional matching = integer matching
Kőnig–Egerváry	max matching = min vertex cover
Menger's theorem	max disjoint paths = min separator

All are facets of the same duality: **what you can push through a network equals what it takes to block it**.

Questions?